Chapter 5

APPLET PROGRAMMING

Applets

- **Applets** are small Java programs that are primarily used in internet programming.
- They can be transported over the internet from one computer to another and run using the *appletviewer* or any *web browser* that supports Java.
- Java applet can perform arithmetic operations, display graphics, play sounds, accept user input, create animation and play interactive games.

Types of applets

1) Local applet:

- An applet developed locally and stored in a local system is known as local applet.

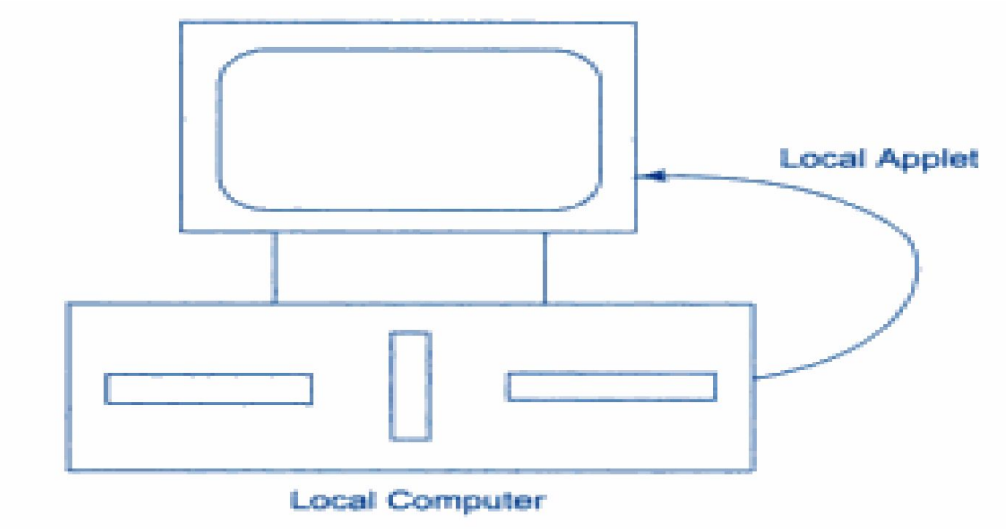
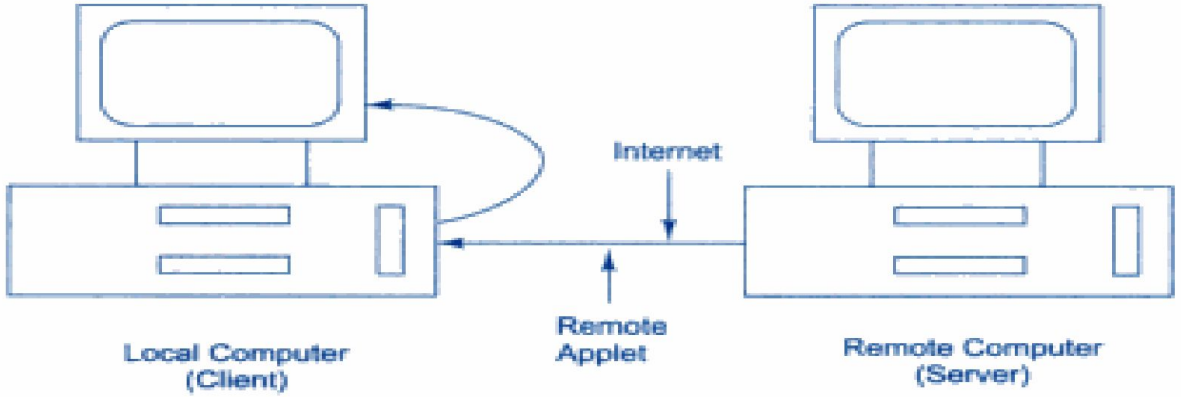


Fig.: Loading local applets

Types of applets

2) **Remote applet:**

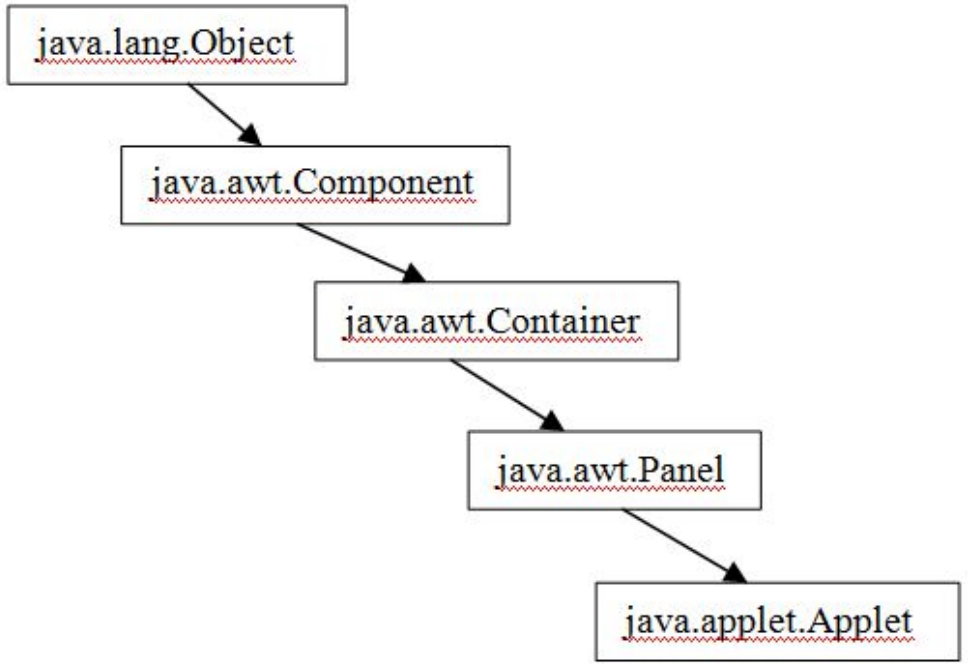
- A remote applet is that which is developed by someone else and stored on a remote computer connected to internet.
- If system is connected to the internet, we can download remote applet onto our system via internet and run it.



Applets and Applications(Difference)

Applet	Application
Applet does not use main() method for initiating execution of code	Application use main() method for initiating execution of code
Applet cannot run independently	Application can run independently
Applet cannot read from or write to files in local computer	Application can read from or write to files in local computer
Applet cannot communicate with other servers on network	Application can communicate with other servers on network
Applet cannot run any program from local computer.	Application can run any program from local computer.
Applet are restricted from using libraries from other language such as C or C++	Application are not restricted from using libraries from other language

Hierarchy of applet class



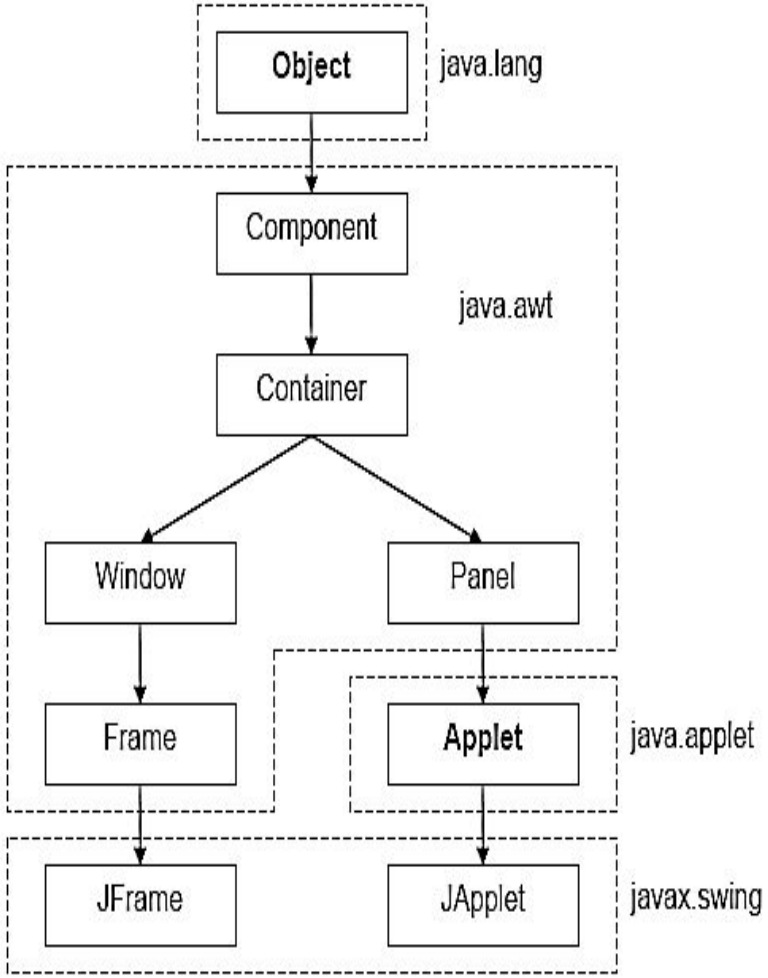


Fig. 6.1 the Applet class hierarchy

Applet Life Cycle

The applet states include:

- **Born or initialization state**
- **Running state**
- **Idle state**
- **Dead or destroyed state**
- **Display state**

Applet Life Cycle

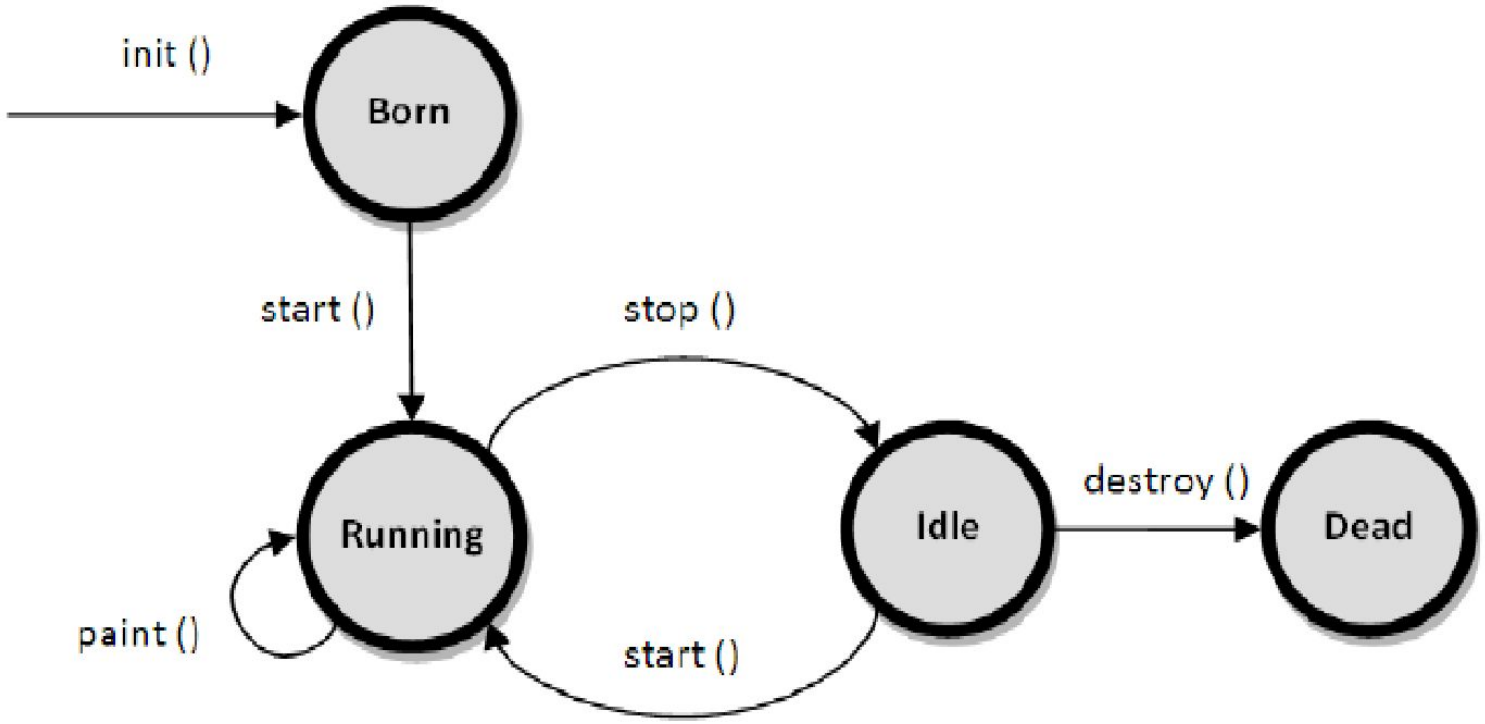


Fig: An Applet's state transition diagram

Initialization state

- Applet enters the initialization state when it is first loaded. This is done by calling the init() method of Applet class. At this stage the following can be done:
  - Create objects needed by the applet
  - Set up initial values
  - Load images or fonts
  - Set up colors

Syntax: public void init()

```
{
//implementation
}
```

Running state

- Applet enters the running state when the system calls the start() method of Applet class.
- This occurs automatically after the applet is initialized.
- Method start() can also be called if the applet is already in idle state.
- **Syntax:**

```
 public void start()
{
  //implementation
}
```

Idle or Stopped state

- An applet becomes idle when it is stopped from running.
- Stopping occurs automatically when the user leaves the page containing the currently running applet.
- **stop()** method may be overridden to terminate the thread used to run the applet.

  Syntax: public void stop()

```
{
  //implementation
}
```

Dead state

- An applet is dead when it is removed from memory.
- This occurs automatically by invoking the destroy() method when we quit the browser.
- Destroying stage occurs only once in the lifetime of an applet.

Syntax: public void destroy()

```
{
   //implementation
}
```

Display State

- Applet is in the display state when it has to perform some output operations on the screen.
- This happens after the applet enters the running state. paint() method is called for this.
- If anything is to be displayed the paint() method is to be overridden.

Syntax:

```
public void paint(Graphics g)
{
    //implementation
}
```

Preparing to write Applets

The steps involved in developing and testing an applet are:

1. Building an applet code(.java file)
2. Creating an executable applet(.class file)
3. Designing a web page using HTML tags
4. Preparing <applet> tag
5. Adding <applet> tag into web page
6. Running the applet.

1) Building an applet code:

- Every applet code should use the services of two classes namely Applet and Graphics.
- The paint() method of Applet class displays the result of the applet code on the screen.
- The output may be text, graphics or sound.

Syntax:

```
import java.awt.*;
import java.applet.*;
public class AppletClassName extends
Applet
{
    ................
    public void paint(Graphics g)
    {
        // Applet code
    }
    ..................
}
```

Display message on applet

- drawString() method used

- It is method of graphics class and use in paint method

Syntax….

public void drawString (String message, int x, int y)

- Message: a string that to display
  x: Horizontal starting position
  y: Vertical Starting position

Applet Example

```
import java.awt.*;
import java.applet.*;
public class HelloJava extends Applet
{
   public void paint(Graphics g)
   {
    g.drawString("Hello Java", 10, 100);
   }
}
```

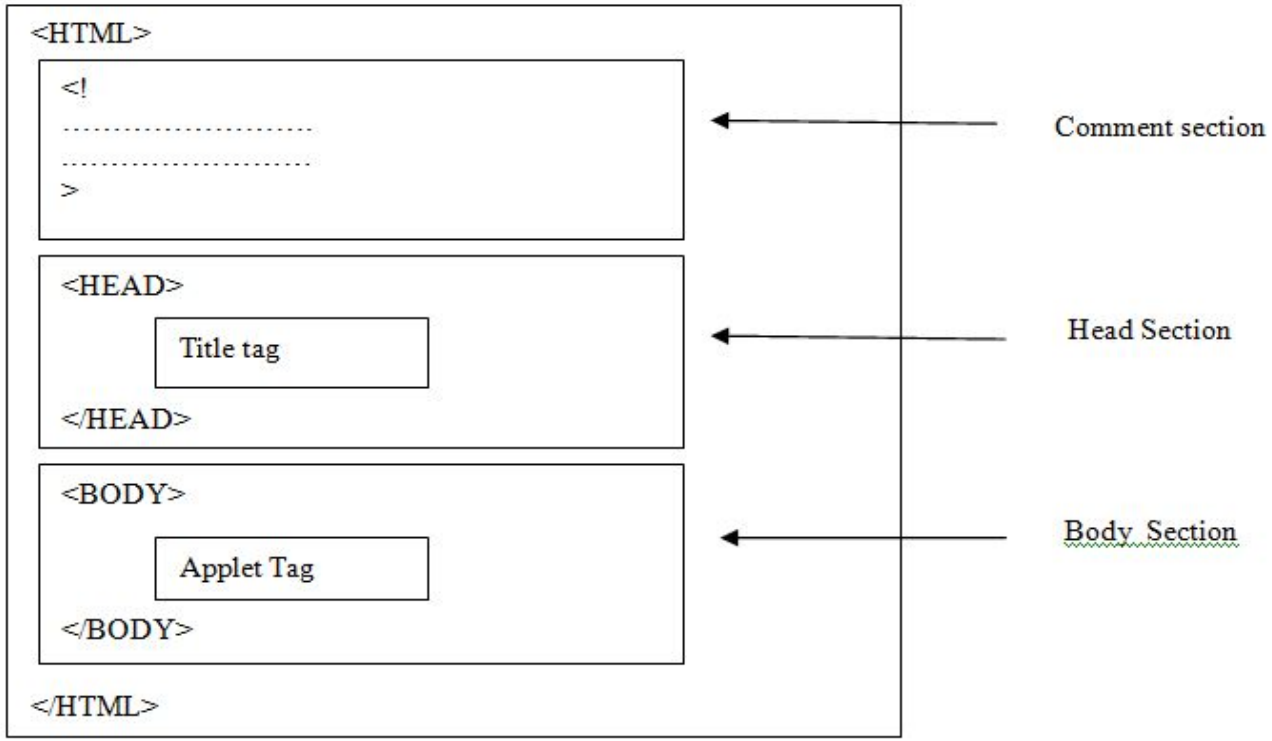
The above applet draws the string "**Hello Java**" at the position 10, 100(pixels) of the applet's reserved space

2) Creating an executable applet

- Executable applet is nothing but the **.class** file of the applet which is obtained by compiling the source code of the applet.
- Compiling an applet is exactly the same as compiling an application.
- C:\Java_Prog>javac HelloJava.java

3) Designing a web page:

- In order to run a java applet, it is first necessary to have a web page that references that applet.
- A typical syntax of web page is shown in following fig:



4) Preparing applet tag:

- The <applet> tag supplies the name of the applet to be loaded and tells the browser how much space applet requires.
  <APPLET CODE = HelloJava.class WIDTH=400 HEIGHT=200>
  </APPLET>
- This HTML code tells the browser to load the compiled Java applet HelloJava.class which is in the same directory as the HTML file.

5) Adding <applet> tag into web page

- Insert the <applet> tag in the page at the space where the output of the applet must appear.
- Following is the contents of HTML file that is embedded with <applet> tag of our applet.

```
<HTML>
    <! Java Applet >
<HEAD>
    <TITLE> Welcome to java applet
</TITLE>
</HEAD>
<BODY>
<APPLET CODE = HelloJava.class
        WIDTH=400
        HEIGHT=200>
</APPLET>
</BODY>
</HTML>
```

6) Running the applet

To run the applet we require one of the following tools:

1) Java-enabled web browser(such as HotJava or Netscape)

- Using this method we will be able to see the entire web page containing the applet.

2) Java appletviewer

- Using this we will only see the applet output.
- The *appletviewer* is available as a part of Java Development kit(JDK) that we have been using so far.
- We can use it to run our applet as follows:

  appletviewer HelloJava.html

6) Running the applet

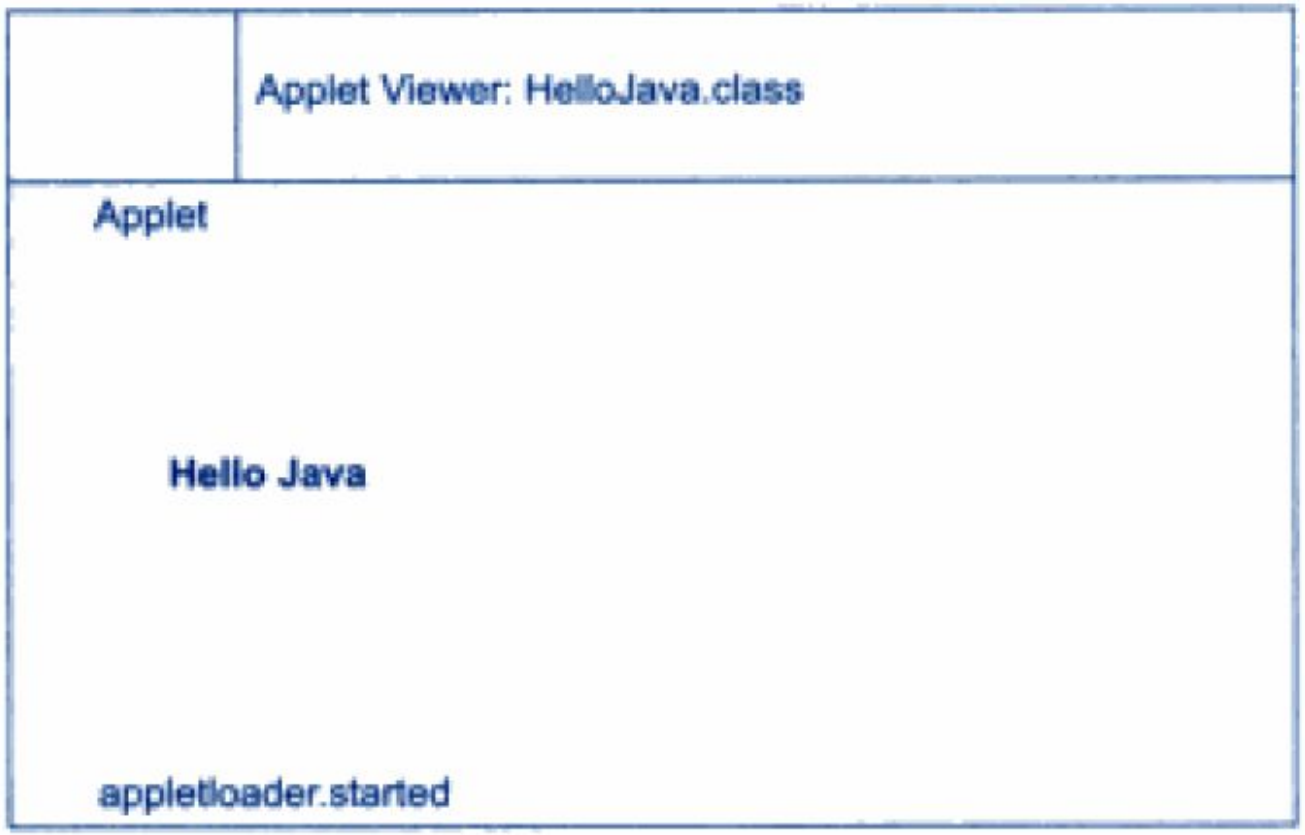


Fig.: Output of HelloJava applet by using appletviewer

Procedure to Run First Applet Program

1. import two package into applet program Namely
   a) java.awt.*; :- abstract window toolkit(act an container to hold the applet viewer)
   b) java.applet.*; : -which define Applet class
2. Write a applet code
3. Write an <applet tag> (considering height and width of applet viewer inside the comments.)
4. Save this file “class_name.java”
5. Compile “c:\>javac class_name.java”
6. Execute “c:\>appletviewer class_name.java”

Applet Methods

1. void setBackground(Color newColor)
2. void setForeground(Color newColor)
3. Color getBackground( )
4. Color getForeground( )

Color.black
Color.blue
Color.cyan
Color.darkGray
Color.gray
Color.green
Color.lightGray
Color.magenta
Color.orange
Color.pink
Color.red
Color.white
Color.yellow

Applet Methods

3. String getParameter(String paramName)

4. void resize(Dimension dim)

5. void resize(int width, int height)

6. void showStatus(String str)

7. void repaint( )

8. void repaint(int left, int top, int width, int height)

9. void repaint(long maxDelay)

10 void repaint(long maxDelay, int x, int y, int width, int height)

Working with Graphics

1. void drawLine(int startX, int startY, int endX, int endY)

2. void drawRect(int top, int left, int width, int height)

3. void fillRect(int top, int left, int width, int height)

4. void drawRoundRect(int top, int left, int width, int height, int xDiam, int yDiam)

5. void fillRoundRect(int top, int left, int width, int height, int xDiam, int yDiam)

Working with Graphics

6. void drawOval(int top, int left, int width, int height)

7. void fillOval(int top, int left, int width, int height)

8. void drawArc(int top, int left, int width, int height, int startAngle, int sweepAngle)

9. void fillArc(int top, int left, int width, int height, int startAngle, int sweepAngle)

10. void drawPolygon(int x[ ], int y[ ], int numPoints)

11.void fillPolygon(int x[ ], int y[ ], int numPoints)

Setting Color

12. void setColor(Color newColor)

13. Color getColor()

The HTML Applet Tag

< APPLET
[CODEBASE = *codebaseURL*]
CODE = *appletFile.class*
[ALT = *alternateText*]
[NAME = *appletInstanceName*]
WIDTH = *pixels*
HEIGHT = *pixels*
[ALIGN = *alignment*]
[VSPACE = *pixels*]
[HSPACE = *pixels*]
>
[< PARAM NAME = *AttributeName1* VALUE = *AttributeValue*>]
[< PARAM NAME = *AttributeName2* VALUE = *AttributeValue*>]
. . .
[*HTML Displayed in the absence of Java*]
</APPLET>

Passing Parameters to Applets

- To pass parameters to an applet <PARAM… > tag is used.
- Each <PARAM…> tag has a **name** attribute such as *color* and a **value** attribute such as *red*. Inside the applet code, the applet can refer to that parameter by name to find its value.
- The syntax of <PARAM…> tag is as follows:

<PARAM NAME = name1 VALUE = value1>

e.g: <PARAM NAME = *text* VALUE = *"I love Java"*>

Passing Parameters to Applets

- To set up and handle parameters, two things must be done.
  1. Include appropriate <PARAM..> tags in the HTML document.
  2. Provide code in the applet to parse these parameters.

- Generally **init()** method in the applet is used to get hold of the parameters defined in the <PARAM…> tag.

- The **getParameter()** method, which takes one string argument representing the name of the parameter and returns a string containing the value of that parameter.

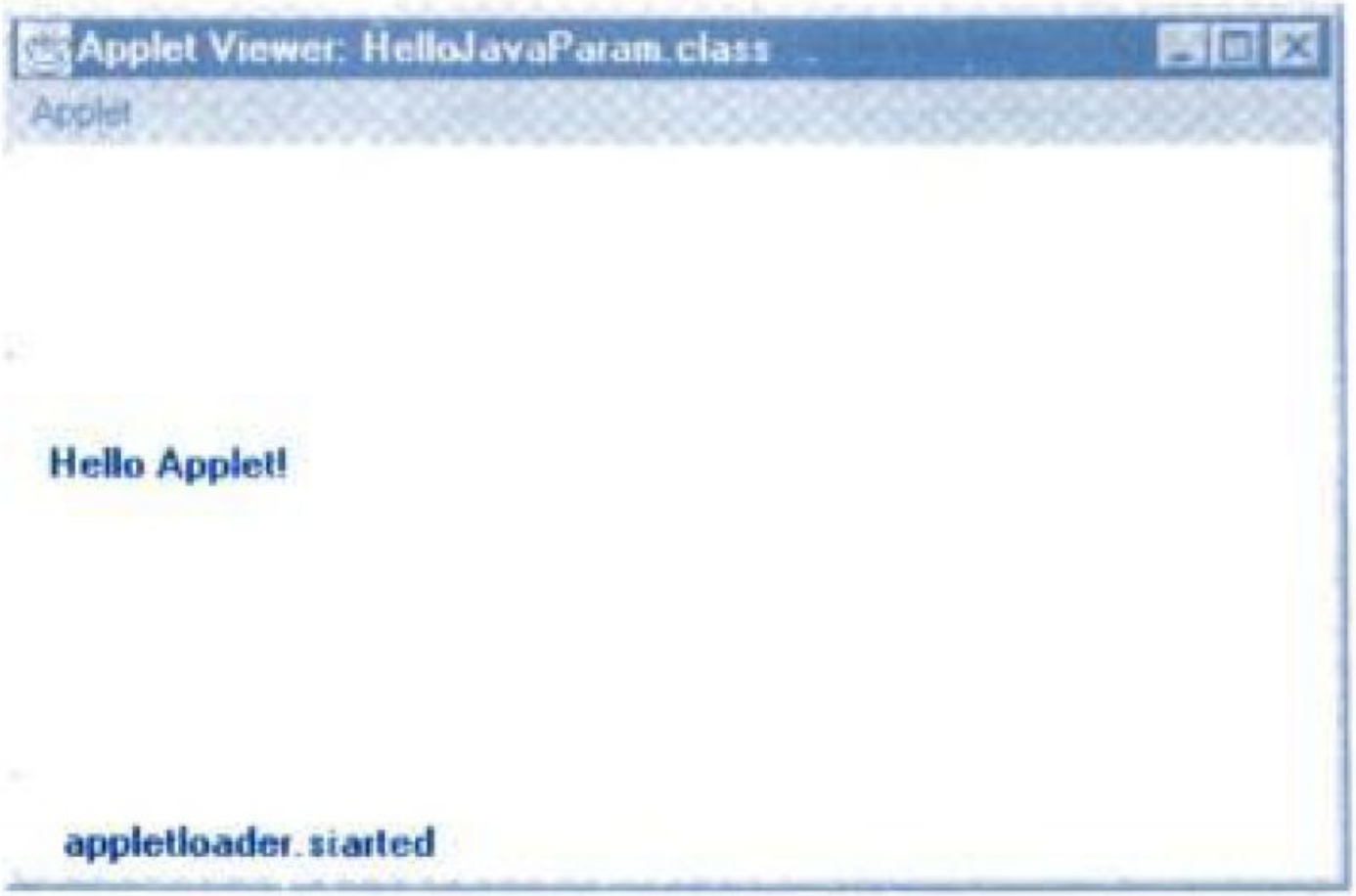
Passing Parameters to Applets

```
import java.awt.*;
import java.applet.*;
public class HelloJavaParam extends Applet
{
String str;
public void init()
{
str = getParameter("string");
str = "Hello "+ str;
}
public void paint(Graphics g)
{
g.drawString(str,10,100);
}  }
```

Passing Parameters to Applets

```
<HTML>
   <!Parameterized HTML file>
   <HEAD>
   <TITLE> Welcome to Java Applet </TITLE>
   </HEAD>
   <BODY>
<Applet code = HelloJavaParam.class width = 400 height = 400>
<PARAM NAME = "string" VALUE =" APPLET!">
</Applet>
   </BODY>
</HTML>
```

Passing Parameters to Applets



Applet Methods

1. URL getDocumentBase()

2. URL getCodeBase()

3. void showDocument(URL url)

Working with Color

Color(int red, int green, int blue)
Color(int rgbValue)
Color(float red, float green, float blue)

int getRed( )
int getGreen( )
int getBlue( )

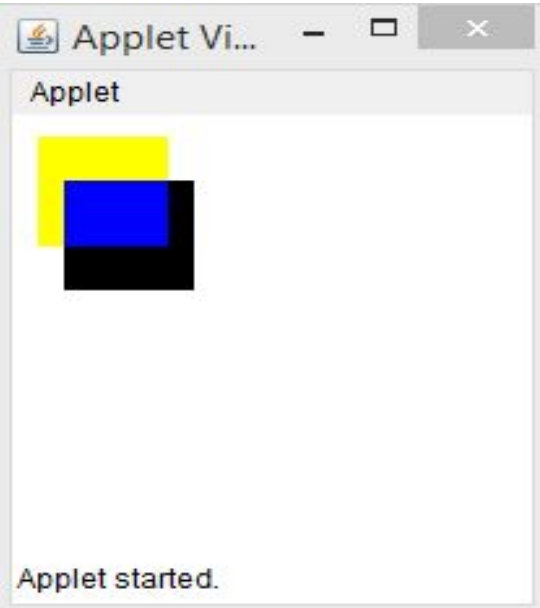
Setting the Paint Mode

```
void setXORMode(Color xorColor)

void setPaintMode( )
```

```
import java.applet.Applet;
import java.awt.Color;
import java.awt.Graphics;
```

```
 /*<applet code="SetXORPaintModeExample" width=200 height=200>
    </applet>  */
public class SetXORPaintModeExample extends Applet
{
 public void paint(Graphics g)
  {
    g.setColor(Color.yellow);
    g.fillRect(10,10,50,50);
    g.setXORMode(Color.blue);
    g.fillRect(20,30,50,50);
    g.setPaintMode();
 }
}
```



Working with Fonts

Font(String fontName, int fontStyle, int pointSize)

<u>Constants : Font.PLAIN, Font.BOLD, Font.ITALIC</u>

- void setFont(Font fontObj)
- Font getFont()

- String getFamily()
- String getName()
- int getSize()
- int getStyle()
- boolean isBold() boolean isItalic() boolean isPlain()

```
import java.applet.Applet;
import java.awt.*;

/* <APPLET CODE ="FontClass.class" WIDTH=300 HEIGHT=200> </APPLET> */

public class FontClass extends Applet  {
        Font f;
        String m;
          public void init()
            {
                f=new Font("Arial",Font.ITALIC,20);
                m="Welcome to Java";
                setFont(f);
            }
                public void paint(Graphics g)
                    {
                        Color c=new Color(0,255,0);
                        g.setColor(c);
                        g.drawString(m,4,20);
                    }
  }
```